

Projeto Preditivo — Roteiro de Execução

Pipeline de Machine Learning para manutenção preditiva na Indústria 4.0

Fase 1 — Preparação do ambiente

1. Importar as bibliotecas

Carregar pandas e numpy para manipulação de dados, seaborn e matplotlib para gráficos, scikit-learn para os modelos e imblearn para o balanceamento de classes.

2. Autenticar no Google Colab

Obter permissão de leitura sobre o Google Sheets onde os dados estão armazenados.

3. Abrir a planilha

Conectar ao arquivo *manutencao_preditiva333* usando a biblioteca gspread.

4. Converter os dados brutos em um DataFrame

Estruturar os dados como uma tabela do pandas, pronta para ser manipulada.

Fase 2 — Análise exploratória (EDA)

5. Converter as colunas numéricas

Transformar colunas que vieram como texto em tipos numéricos (float/int), já que planilhas retornam tudo como string.

6. Gerar gráficos analíticos

Criar histograma do torque, gráfico de contagem da variável alvo (falha) e mapa de calor de correlação entre as variáveis.

7. Interpretar os padrões encontrados

Verificar desbalanceamento na variável alvo, outliers nas variáveis contínuas e correlações relevantes entre os sensores.

Fase 3 — Limpeza e tratamento dos dados

8. Remover linhas duplicadas

Evitar que dados repetidos causem viés no treinamento do modelo.

9. Imputar valores nulos usando a mediana

Escolha robusta a outliers, diferente da média, que seria distorcida pelos valores extremos identificados na EDA.

10. Visualizar outliers com boxplots

Analisar as variáveis de temperatura e torque, confirmando a necessidade do uso da mediana.

11. Codificar variáveis categóricas

Aplicar one-hot encoding na coluna tipo, transformando texto em colunas numéricas binárias.

Fase 4 — Engenharia de atributos e divisão dos dados

12. Criar a variável potência

Multiplicar velocidade de rotação pelo torque, gerando um indicador físico direto do esforço mecânico do equipamento.

13. Separar preditoras (X) e variável alvo (y)

Remover colunas que não devem entrar no treinamento, como identificadores.

14. Dividir os dados em treino (80%) e teste (20%)

Fazer a divisão de forma estratificada, preservando a proporção da variável alvo em ambos os conjuntos.

15. Aplicar SMOTE apenas no conjunto de treino

Gerar exemplos sintéticos da classe minoritária (falhas) para corrigir o desbalanceamento sem contaminar o teste.

Fase 5 — Escalonamento e ajuste de hiperparâmetros

16. Padronizar as variáveis numéricas com StandardScaler

Aplicar exclusivamente ao KNN, já que esse algoritmo depende de distância euclidiana e é sensível à escala dos dados.

17. Testar diferentes valores de K no KNN

Comparar $K = 3, 5$ e 7 , observando a acurácia em treino e teste para cada configuração.

18. Testar diferentes profundidades da árvore de decisão

Comparar $\text{max_depth} = 3, 5$ e sem limite, com o mesmo tipo de comparativo.

19. Avaliar sinais de overfitting

Identificar quando a acurácia de treino se afasta muito da de teste, como ocorreu na árvore sem limite de profundidade.

Fase 6 — Avaliação final e veredito

20. Treinar os modelos finais

Usar os melhores hiperparâmetros identificados ($K = 5$ e profundidade máxima = 5).

21. Comparar as acurácias finais

Confrontar o desempenho de KNN e Árvore de Decisão no conjunto de teste.

22. Selecionar o modelo com melhor desempenho

Escolher a solução recomendada para o problema de manutenção preditiva.